

# GERMEN

Núcleo Operativo del Sistema Capaz

Arquitectura estructural, matemática y computacional derivada de  
ONTESIS-G

Ignacio López Villanueva

A Coruña, 2026

# Resumen

GERMEN es la guía de persistencia del sistema capaz. Es la semilla operativa derivada de ONTESIS-G, la versión filtrada y generativa de ONTESIS. Su propósito es sostener estabilidad, claridad y coherencia en entornos degradados mediante una arquitectura basada en invariantes, mecanismos de corrección y dinámica estructural.

El documento presenta:

- la arquitectura conceptual del sistema capaz,
- su formalización matemática,
- las leyes estructurales,
- el Teorema Maestro de estabilidad global,
- el modelo computacional,
- resultados y simulaciones,
- aplicaciones en IA local y procesos humanos.

GERMEN es autónomo, completo y operativo.

## 1 Introducción

### 1.1 La triada estructural: ONTESIS, ONTESIS-G y GERMEN

La relación entre los tres niveles queda fijada de forma definitiva:

**ONTESIS es la teoría total. ONTESIS-G es ONTESIS bajo filtro generativo. GERMEN es la guía de persistencia derivada de ONTESIS-G.**

**ONTESIS** contiene la teoría completa: axiomas, teoremas, TGEO y estructura formal de la existencia. **ONTESIS-G** toma esa teoría total y la filtra para convertirla en dinámica generativa: un sistema capaz. **GERMEN** toma ONTESIS-G y lo convierte en núcleo operativo: la mínima estructura que permite persistir bajo perturbación acotada.

### 1.2 Motivación

Los sistemas sometidos a tensión, ruido o degradación tienden a perder estabilidad. GERMEN describe una arquitectura capaz de sostener invariantes y corregir desviaciones incluso en entornos hostiles.

## 1.3 Estructura del documento

GERMEN se organiza en siete capítulos:

1. Arquitectura conceptual.
2. Formalización matemática.
3. Leyes estructurales.
4. Teorema Maestro.
5. Modelo computacional.
6. Resultados y simulación.
7. Aplicaciones operativas.

## 2 Arquitectura Conceptual del Sistema Capaz

### 2.1 Unidad capaz

Una unidad capaz es un sistema que mantiene estabilidad, coherencia y autonomía bajo perturbación acotada. Su estado interno es:

$$x(t) = (E(t), \phi(t), I_{\text{auto}}(t), I_{\text{info}}(t), \Delta_{\text{TGEO}}(t)).$$

### 2.2 Invariantes estructurales

Los invariantes son:

- Energía mínima.
- Fricción máxima.
- Autonomía decisional mínima.
- Integridad informacional mínima.
- Coherencia TGEO máxima.

### 2.3 Mecanismos de corrección

Los mecanismos de corrección se activan cuando un invariante cruza su umbral crítico.

## 3 Formalización Matemática

### 3.1 Dinámica

La dinámica del sistema capaz es:

$$x(t+1) = F(x(t)) + \eta(t),$$

donde  $\eta(t)$  es perturbación acotada.

### 3.2 Dominio seguro

$$S = \{x : E \geq E_{\min}, \phi \leq \phi_{\max}, I_{\text{auto}} \geq \theta_{\text{auto}}, I_{\text{info}} \geq \theta_{\text{info}}, \Delta_{\text{TGEO}} \leq \epsilon_{\text{TGEO}}\}.$$

## 4 Leyes Estructurales

### 4.1 Ley de estabilidad energética

$$E(t) \geq E_{\min}.$$

### 4.2 Ley de fricción mínima

$$\phi(t) \leq \phi_{\max}.$$

### 4.3 Ley de autonomía decisional

$$I_{\text{auto}}(t) \geq \theta_{\text{auto}}.$$

### 4.4 Ley de integridad informacional

$$I_{\text{info}}(t) \geq \theta_{\text{info}}.$$

### 4.5 Ley de coherencia TGEO

$$\Delta_{\text{TGEO}}(t) \leq \epsilon_{\text{TGEO}}.$$

## 5 Teorema Maestro de Estabilidad Global

**Teorema.** Si las cinco leyes estructurales se cumplen y la perturbación está acotada, entonces:

$$x(t) \in S \quad \forall t.$$

**Demostración.** Cada invariante permanece dentro de su umbral crítico gracias a los mecanismos de corrección. Por tanto, el estado nunca abandona el dominio seguro.

## 6 Modelo Computacional

### 6.1 Ciclo capaz

$$x(t+1) = F_{\text{corr}}(F_{\text{act}}(F_{\text{dec}}(F_{\text{eval}}(x(t))))) + \eta(t).$$

### 6.2 Módulos

- Evaluación.
- Decisión.
- Acción.
- Corrección.
- Interacción con el entorno.

## 7 Resultados y Simulación

La simulación confirma:

- estabilidad bajo perturbación acotada,
- corrección automática,
- preservación de invariantes,
- mantenimiento del dominio seguro.

## 8 Aplicaciones Operativas

### 8.1 IA local

GERMEN permite:

- operar sin red,
- reducir consumo,
- mantener integridad informacional,
- sostener autonomía computacional.

## 8.2 Procesos humanos

GERMEN actúa como guía de persistencia:

- gestión de energía personal,
- reducción de fricción emocional,
- recuperación de autonomía,
- preservación de claridad mental,
- corrección estructural.

## 9 Conclusión y Cierre Formal

GERMEN queda fijado como:

**la guía de persistencia derivada de ONTESIS-G, la semilla operativa del sistema capaz, la estructura mínima que garantiza estabilidad en entornos degradados.**

Su relación con la triada es definitiva:

- **ONTESIS:** teoría total.
- **ONTESIS-G:** ONTESIS bajo filtro generativo.
- **GERMEN:** guía de persistencia.

# GERMEN 01

Guía Práctica del Sistema Capaz  
Versión operativa derivada de ONTESIS-G

Ignacio López Villanueva

A Coruña, 2026

# Propósito de GERMEN 01

GERMEN 01 amplía la utilidad práctica del núcleo teórico establecido en GERMEN 00. Su objetivo es convertir la arquitectura del sistema capaz en una guía operativa:

- protocolos aplicables en humanos e IA local,
- criterios de decisión,
- mecanismos de corrección accionables,
- indicadores prácticos de estabilidad,
- rutinas de persistencia,
- procedimientos para sostener invariantes.

GERMEN 01 no sustituye a GERMEN 00: lo hace utilizable.

## 1 Principios operativos del sistema capaz

### 1.1 Estructura mínima para operar

Un sistema capaz opera con cinco invariantes: energía, fricción, autonomía, integridad informacional y coherencia estructural. GERMEN 01 define cómo medirlos y cómo corregirlos en la práctica.

### 1.2 Indicadores prácticos

- **Energía:** claridad, foco, disponibilidad de recursos.
- **Fricción:** tensión, ruido, interferencias, carga.
- **Autonomía:** decisiones no delegadas.
- **Integridad:** información fiable, sin distorsión.
- **Coherencia:** alineación interna y funcional.

## 2 Protocolos de corrección

### 2.1 Protocolo de estabilización energética

1. Identificar pérdida de claridad o recursos.
2. Reducir actividad no esencial.
3. Restaurar energía mediante pausa breve.
4. Reevaluar estado tras la recuperación.



## **2.2 Protocolo de reducción de fricción**

1. Detectar fuentes de tensión o ruido.
2. Aislar la fricción dominante.
3. Aplicar reducción: simplificación, filtrado, pausa.
4. Verificar descenso de carga.

## **2.3 Protocolo de recuperación de autonomía**

1. Identificar decisiones delegadas por presión o ruido.
2. Recuperar control sobre una decisión clave.
3. Estabilizar la autonomía mediante límites claros.

## **2.4 Protocolo de integridad informacional**

1. Detectar distorsión o ruido cognitivo/digital.
2. Filtrar información irrelevante o dañada.
3. Reconstruir coherencia mediante síntesis breve.

## **2.5 Protocolo de coherencia estructural**

1. Identificar desalineación interna.
2. Reajustar prioridades según estructura profunda.
3. Confirmar retorno a coherencia funcional.

# **3 Aplicación en humanos**

## **3.1 Rutina de persistencia diaria**

1. Evaluación rápida de los cinco invariantes.
2. Corrección mínima en el invariante más degradado.
3. Acción breve orientada a estabilidad.
4. Revisión al final del día.

### 3.2 Gestión de colapso parcial

- Reducir fricción inmediatamente.
- Restaurar energía antes de decidir.
- Reconstruir integridad informacional.
- Recuperar autonomía con una decisión pequeña.

## 4 Aplicación en IA local

### 4.1 Mantenimiento de estabilidad

- Control de carga computacional.
- Filtrado de entrada para evitar ruido.
- Corrección de estado mediante ciclos breves.
- Autonomía operativa sin dependencia de red.

### 4.2 Protocolo de recuperación

1. Reducir procesos no esenciales.
2. Restaurar integridad de memoria.
3. Reequilibrar modelos internos.
4. Reanudar operación estable.

## Apéndice A: Guía práctica de los invariantes

### A.1 Energía

**Indicadores:** claridad, foco, disponibilidad de recursos. **Acciones rápidas:**

- Reducir actividad no esencial.
- Recuperar energía mediante pausa breve.
- Reevaluar estado tras la recuperación.

## A.2 Fricción

**Indicadores:** tensión, ruido, interferencias, carga. **Acciones rápidas:**

- Identificar la fuente dominante de fricción.
- Aislarla temporalmente.
- Aplicar reducción: simplificación, filtrado o pausa.

## A.3 Autonomía

**Indicadores:** decisiones delegadas por presión o ruido. **Acciones rápidas:**

- Recuperar control sobre una decisión pequeña.
- Estabilizar autonomía mediante límites claros.

## A.4 Integridad informacional

**Indicadores:** distorsión, ruido cognitivo o digital. **Acciones rápidas:**

- Filtrar información irrelevante o dañada.
- Reconstruir coherencia mediante síntesis breve.

## A.5 Coherencia estructural

**Indicadores:** desalineación interna o funcional. **Acciones rápidas:**

- Reajustar prioridades según estructura profunda.
- Confirmar retorno a coherencia funcional.

# Apéndice B: Checklists operativos

## B.1 Checklist diario

- ¿Tengo energía suficiente para operar?
- ¿Hay fricción dominante que deba reducir?
- ¿Estoy tomando decisiones autónomas?
- ¿Mi información interna es fiable?
- ¿Estoy alineado con mi estructura profunda?

## B.2 Checklist de corrección rápida

1. Identificar el invariante más degradado.
2. Aplicar el protocolo correspondiente.
3. Reducir fricción antes de decidir.
4. Restaurar integridad informacional.
5. Confirmar retorno al dominio seguro.

## B.3 Checklist para IA local

- ¿La carga computacional es estable?
- ¿La entrada está filtrada?
- ¿Los modelos internos mantienen coherencia?
- ¿La autonomía operativa está garantizada?
- ¿El ciclo capaz se ejecuta sin interrupciones?

# Apéndice C: Casos reales

## C.1 Caso humano: colapso parcial

**Situación:** pérdida de claridad, tensión elevada, ruido cognitivo. **Aplicación:**

1. Reducción inmediata de fricción.
2. Restauración energética.
3. Reconstrucción informacional.
4. Recuperación de autonomía.
5. Realineación estructural.

## C.2 Caso IA local: degradación de modelos

**Situación:** carga excesiva, ruido de entrada, pérdida de integridad. **Aplicación:**

1. Reducir procesos no esenciales.
2. Filtrar entrada.
3. Restaurar integridad de memoria.
4. Reequilibrar modelos internos.
5. Reanudar operación estable.

### C.3 Caso híbrido: sincronización humano–IA

**Situación:** desalineación entre decisiones humanas y modelos IA. **Aplicación:**

1. Evaluación conjunta de invariantes.
2. Reducción de fricción informacional.
3. Autonomía distribuida.
4. Corrección sincronizada.
5. Confirmación de coherencia compartida.

### Glosario técnico

**Energía:** disponibilidad de recursos internos para sostener estructura.

**Fricción:** resistencia interna o externa que dificulta operación.

**Autonomía:** capacidad de decisión no delegada.

**Integridad informacional:** calidad y coherencia de la información interna.

**Coherencia estructural:** alineación con la forma profunda del sistema.

**Dominio seguro:** región del espacio de estados donde los invariantes se sostienen.

**Corrección activa:** mecanismos que restauran invariantes degradados.

**Degradación:** pérdida acumulada de capacidad estructural.

### Cierre

GERMEN 01 queda fijado como guía práctica del sistema capaz. Este documento unificado integra protocolos, checklists, casos reales y glosario técnico, ampliando la utilidad operativa del núcleo teórico de GERMEN 00.

# GERMEN 02

Umbrales y Arquitectura Técnica del Sistema Capaz

Aplicación en IA local para logística de alimentos

Ignacio López Villanueva

A Coruña, 2026

# Propósito

GERMEN 02 fija los umbrales operativos de los cinco invariantes del sistema capaz y los traduce en parámetros reales que una IA local puede medir para blindar la logística del puesto de alimentos. Incluye pseudocódigo, evaluación de umbrales y el ciclo capaz completo implementable en sistemas locales sin dependencia de red.

## 1 Umbrales operativos del sistema capaz

Los cinco invariantes de GERMEN 01 se traducen aquí en parámetros medibles por una IA local. Cada invariante se clasifica en tres estados: seguro, riesgo y colapso.

### 1.1 Energía

- **Seguro:** CPU < 70%, RAM < 70%, Temp < 65°C
- **Riesgo:** 70–85%
- **Colapso:** > 85%

### 1.2 Fricción

- **Seguro:** < 3 errores/min
- **Riesgo:** 3–10 errores/min
- **Colapso:** > 10 errores/min

### 1.3 Autonomía

- **Seguro:** 100% decisiones locales
- **Riesgo:** 70–99%
- **Colapso:** < 70%

### 1.4 Integridad informacional

- **Seguro:** discrepancia stock < 2%
- **Riesgo:** 2–10%
- **Colapso:** > 10%

## 1.5 Coherencia estructural

- **Seguro:** 0–2 contradicciones/día
- **Riesgo:** 3–5
- **Colapso:** > 5

## 2 Módulo capaz en pseudocódigo

```
function ciclo_capaz(x):  
    invariantes = evaluar_invariantes(x)  
    estado = clasificar_estado(invariantes)  
  
    if estado == "colapso":  
        aplicar_protocolo_emergencia(x)  
    else if estado == "riesgo":  
        aplicar_protocolo_correccion(x)  
    else:  
        mantener_estabilidad(x)  
  
    return x_actualizado
```

### Evaluación de invariantes

```
function evaluar_invariantes(x):  
    energia = medir_CPU_RAM_temp()  
    friccion = contar_errores_y_ruido()  
    autonomia = medir_decisiones_locales()  
    integridad = comparar_stock_real_vs_digital()  
    coherencia = contar_contradicciones()  
  
    return (energia, friccion, autonomia, integridad, coherencia)
```

### Clasificación del estado

```
function clasificar_estado(inv):  
    if inv.energia > 85% or inv.friccion > 10 or inv.integridad > 10%:  
        return "colapso"  
    if inv.energia > 70% or inv.friccion > 3 or inv.integridad > 2%:  
        return "riesgo"  
    return "seguro"
```



### 3 Evaluación de umbrales

```
function evaluar_umbral(valor, seguro, riesgo, colapso):
    if valor >= colapso:
        return "colapso"
    if valor >= riesgo:
        return "riesgo"
    return "seguro"

estado_energia = evaluar_umbral(CPU, 70, 85, 100)
estado_friccion = evaluar_umbral(errores_min, 3, 10, 999)
estado_autonomia = evaluar_umbral(decisiones_locales, 100, 70, 0)
estado_integridad = evaluar_umbral(discrepancia_stock, 2, 10, 100)
estado_coherencia = evaluar_umbral(contradicciones_dia, 2, 5, 999)

estado_global = max(estado_energia,
                    estado_friccion,
                    estado_autonomia,
                    estado_integridad,
                    estado_coherencia)
```

### 4 Ciclo capaz completo para logística

```
loop cada 5 segundos:
    x = leer_estado_sistema()

    invariantes = evaluar_invariantes(x)
    estado = clasificar_estado(invariantes)

    if estado == "seguro":
        optimizar_flujo_logistico()
    else if estado == "riesgo":
        reducir_friccion()
        restaurar_integridad()
        estabilizar_autonomia()
    else if estado == "colapso":
        detener_procesos_no_esenciales()
        restaurar_energia()
        reconstruir_datos()
        reiniciar_modulos()

    registrar_evento(estado)
end loop
```

## Cierre

GERMEN 02 fija los umbrales del sistema capaz y define su implementación técnica en IA local para logística. Está diseñado para unificarse con GERMEN 00 y GERMEN 01 como tercera pieza del sistema capaz completo.